

CESUR CARTUJA — SEVILLA

Ciclo Formativo de Grado Superior, Desarrollo de Aplicaciones Multiplataforma
(DAM)

PROYECTO FIN DE MÓDULO INTERMODULAR

NutriFit

Sistema de seguimiento nutricional y deportivo

Autor:

Antonio Manuel Fresco Gómez

Curso: 2.º DAM · Año académico 2025/2026

<https://github.com/AntManFresc2006/NutriFit>

Mayo de 2026

Índice

1. Introducción y contexto
 1. Problema que aborda el proyecto
 2. Objetivo del proyecto
 3. Alcance del sistema
 4. Arquitectura y tecnologías
 5. Organización de la memoria

2. Estado del arte y marco teórico
3. Tecnologías y herramientas
4. Análisis de requisitos
5. Diseño del sistema
6. Implementación
7. Pruebas
8. Seguridad
9. Conclusiones y líneas futuras
10. Bibliografía

Anexo A — Guía de puesta en marcha

Anexo B — Referencia de endpoints de la API

1. Introducción y contexto

El seguimiento de la alimentación es una práctica habitual en contextos de salud, deporte y gestión del peso. Registrar qué se come, cuánto y cuándo da a una persona información objetiva sobre su ingesta calórica y su distribución de macronutrientes a lo largo del día. Sin ese registro, cualquier ajuste dietético se basa en estimaciones.

Existen herramientas comerciales que cubren esta necesidad, pero su complejidad interna queda opaca al desarrollador que quiera aprender de ellas. NutriFit parte de la misma necesidad funcional y la aborda como ejercicio de ingeniería: construir desde cero un sistema multicapa que gestione alimentos, registre comidas, calcule resúmenes nutricionales diarios, integre inteligencia artificial para evaluar la dieta, registre ejercicios y estime el gasto energético del usuario, con decisiones técnicas razonadas y documentadas en cada paso.

1.1 Problema que aborda el proyecto

El sistema responde a una necesidad concreta: que un usuario pueda registrar los alimentos que consume durante el día, organizarlos por comidas, obtener un resumen del total calórico y de macronutrientes, y recibir una evaluación personalizada de su dieta basada en inteligencia artificial.

El problema no es complejo en su enunciado, pero su implementación involucra decisiones no triviales: cómo modelar la relación entre comidas e ítems, cómo calcular los valores nutricionales sin mover datos innecesariamente entre capas, cómo gestionar sesiones con logout real, cómo evitar que un usuario acceda a los datos de otro, cómo integrar modelos de lenguaje sin acoplar la lógica de negocio a un proveedor concreto, o cómo garantizar que el esquema de base de datos es reproducible en cualquier entorno. Esas decisiones son el centro del trabajo.

1.2 Objetivo del proyecto

El objetivo principal es diseñar, implementar y documentar un sistema completo de seguimiento nutricional con tres interfaces de usuario: una aplicación web (React SPA), un cliente de escritorio (JavaFX) y una API REST como capa central. La versión web está desplegada en producción y es accesible públicamente.

El objetivo no es construir una aplicación comercial lista para escalar a millones de usuarios, sino demostrar la capacidad de tomar decisiones técnicas justificadas, implementarlas de forma coherente dentro de un alcance definido y documentar el resultado con precisión. Esto incluye reconocer las limitaciones del sistema y distinguir con claridad qué está hecho, qué queda fuera del alcance y por qué.

1.3 Alcance del sistema

NutriFit cubre dieciséis áreas funcionales en su versión actual:

- **Autenticación:** registro, login y logout con invalidación inmediata del token en el servidor.
- **Catálogo de alimentos:** creación, consulta, búsqueda por nombre, actualización y eliminación, con valores nutricionales por 100 gramos.
- **Escáner de alimentos:** búsqueda por código de barras EAN en Open Food Facts y, de forma complementaria, análisis de una foto del producto mediante un modelo de visión para estimar sus macronutrientes.
- **Registro de comidas:** comidas asociadas a una fecha y tipo de toma, con adición y eliminación de ítems en gramos.
- **Resumen diario:** total de kilocalorías, proteínas, grasas e hidratos de carbono consumidos en una fecha, junto al balance frente al gasto estimado.
- **Evaluación nutricional con IA:** análisis del resumen diario mediante un modelo de lenguaje (OpenRouter), con contexto de los últimos siete días.
- **Perfil biométrico:** sexo, fecha de nacimiento, altura, peso y nivel de actividad; cálculo automático de TMB y TDEE.
- **Registro de ejercicios:** sesiones aeróbicas y anaeróbicas con cálculo de calorías quemadas mediante factor MET.
- **Hidratación:** registro diario de agua consumida frente a un objetivo de dos litros.
- **Historial de peso:** seguimiento del peso corporal a lo largo del tiempo.
- **Plan semanal:** generación de un plan de comidas personalizado mediante IA, de forma asíncrona.
- **Retos:** desafíos de fitness con seguimiento de progreso e inscripción del usuario.
- **Gamificación:** racha de días activos y puntuación nutricional diaria (NutriScore) sobre cuatro criterios.
- **Detective de hábitos:** análisis estadístico del historial reciente que identifica por qué el usuario no alcanza sus objetivos, ampliado con una conclusión generada por IA.
- **Tendencias:** series históricas de peso, NutriScore, macronutrientes y ejercicio de hasta noventa días.
- **Configuración de IA:** cada usuario puede definir su propio modelo, clave de API y, opcionalmente, una URL de proxy validada.

1.4 Arquitectura y tecnologías

El sistema se organiza en tres capas independientes. La capa de presentación tiene dos implementaciones: una aplicación web React desplegada en Vercel y un cliente de escritorio JavaFX. Ambas consumen el mismo backend mediante peticiones HTTP. El backend,

implementado con Spring Boot 3.5, expone una API REST con dieciocho prefijos de endpoint y concentra toda la lógica de negocio y el acceso a datos. PostgreSQL almacena el estado del sistema bajo un esquema gestionado íntegramente por Flyway (veintiocho migraciones).

El acceso a datos se realiza con JdbcTemplate y RowMapper manuales, sin ORM. La autenticación usa tokens opacos UUID v4 almacenados en base de datos y enviados como cookie `HttpOnly`. Los errores están centralizados en un `@RestControllerAdvice` que garantiza una estructura de respuesta uniforme para cualquier excepción.

1.5 Organización de la memoria

La memoria sigue el desarrollo del proyecto de forma progresiva:

- §2 — Estado del arte: revisión de soluciones existentes y marco teórico nutricional.
- §3 — Tecnologías y herramientas: stack completo con justificación de cada elección.
- §4 — Análisis de requisitos: requisitos funcionales y no funcionales.
- §5 — Diseño: arquitectura, esquema de base de datos y decisiones de diseño.
- §6 — Implementación: descripción módulo a módulo.
- §7 — Pruebas: estrategia y suite de 199 tests.
- §8 — Seguridad: medidas implementadas y limitaciones conocidas.
- §9 — Conclusiones y líneas futuras.

2. Estado del arte y marco teórico

2.1 El problema del seguimiento nutricional

El control de la ingesta calórica y la distribución de macronutrientes tiene interés en el ámbito clínico, deportivo y de la salud general. El registro sistemático de la alimentación es uno de los predictores más consistentes de éxito en programas de control de peso, según estudios publicados en revistas de nutrición y medicina preventiva.

El problema técnico es siempre el mismo: un sistema que relacione alimentos con sus valores nutricionales, que permita al usuario registrar qué cantidades ha consumido en qué momento del día, y que agregue esa información para ofrecer una visión del conjunto. Sobre ese núcleo básico se construyen los sistemas más complejos.

2.2 Soluciones comerciales existentes

2.2.1 MyFitnessPal

MyFitnessPal es la aplicación de seguimiento nutricional más utilizada a nivel mundial. Tiene una base de datos de más de 14 millones de alimentos, seguimiento de macros y micros, integración con dispositivos wearables y comunidad activa. Su limitación desde el punto de vista técnico es que opera como una caja negra: el usuario no puede conocer cómo se modelan las comidas internamente ni qué decisiones de diseño subyacen a su API.

2.2.2 Cronometer

Cronometer se diferencia de MyFitnessPal por su énfasis en la precisión nutricional, incluyendo vitaminas, minerales y aminoácidos. Usa bases de datos validadas científicamente. Su interfaz es más densa que la de MyFitnessPal, orientada a usuarios con conocimientos nutricionales previos.

2.2.3 Yazio

Yazio es una aplicación de origen alemán con buena acogida en Europa. Ofrece seguimiento de macros, planes de dieta personalizados y registro de ejercicio. Integra cálculo de TMB y TDEE de forma automática. Su limitación es la dependencia de su propia base de datos de alimentos, que en algunos mercados es menos completa.

2.3 Limitaciones formativas de las soluciones existentes

Estas aplicaciones resuelven el problema del usuario final pero no son útiles como referencia de aprendizaje técnico porque su código fuente no es accesible, no documentan sus decisiones de

modelado y operan a una escala no reproducible en un entorno formativo.

NutriFit aborda el mismo problema funcional con un objetivo diferente: un sistema comprensible, documentado y reproducible, donde cada decisión técnica está justificada y es trazable hasta el código concreto que la implementa.

2.4 Marco teórico: fórmulas y conceptos nutricionales

2.4.1 Fórmula de Mifflin-St Jeor

El cálculo del gasto energético en NutriFit se basa en la fórmula de Mifflin-St Jeor, publicada en 1990 y considerada una de las más precisas para estimar la Tasa Metabólica Basal (TMB) en adultos sanos:

```
Hombres: TMB = (10 × peso_kg) + (6,25 × altura_cm) - (5 × edad) + 5  
Mujeres: TMB = (10 × peso_kg) + (6,25 × altura_cm) - (5 × edad) - 161
```

El Gasto Energético Total Diario (TDEE) se obtiene multiplicando la TMB por el factor de actividad correspondiente: 1,2 para sedentario, 1,375 para actividad ligera, 1,55 para moderada, 1,725 para alta y 1,9 para muy alta.

2.4.2 Factor MET para calorías quemadas

El Equivalente Metabólico (MET) expresa el coste energético de una actividad en relación al metabolismo basal en reposo; un MET equivale a aproximadamente 1 kcal/(kg·h). Para el ejercicio aeróbico, donde el coste depende del tiempo, la fórmula es:

```
kcal = MET × peso_kg × (duración_min / 60)
```

Para el ejercicio anaeróbico (fuerza), donde el tiempo continuo no es la variable relevante, el coste se estima a partir de las series realizadas y un factor de intensidad (baja, media o alta). Los valores MET de referencia están tabulados en el Compendium of Physical Activities de Ainsworth et al. NutriFit incluye en su catálogo más de doscientos ejercicios con su valor MET y su tipo.

2.5 Integración de inteligencia artificial

La versión actual incorpora capacidades de IA a través de OpenRouter, un proxy de API que permite seleccionar entre distintos modelos de lenguaje sin acoplarse a un proveedor. Las funciones de IA en NutriFit son cuatro: la evaluación del resumen nutricional diario, la generación de un plan de comidas semanal, el análisis forense de hábitos del módulo detective y la lectura de una foto de alimento para estimar sus macros. El sistema usa por defecto un modelo principal de Gemma y un modelo de respaldo de otra familia, ambos gratuitos a través de OpenRouter y encadenados con una estrategia de reintento, pero cada usuario puede configurar

su propio modelo y clave; si define además una URL de proxy, esta se valida para impedir que apunte a direcciones internas.

3. Tecnologías y herramientas

3.1 Visión de conjunto

NutriFit está compuesto por tres módulos independientes: **backend** (Spring Boot, Maven), **client** (JavaFX, Maven) y **frontend** (React, Vite). Cada módulo selecciona sus dependencias en función de su responsabilidad; los tres comparten únicamente el protocolo HTTP y el formato JSON.

Tecnología	Versión	Módulo	Papel en el proyecto
Java	21 (LTS)	Backend + cliente	Lenguaje base
Spring Boot	3.5.0	Backend	Marco de aplicación: IoC, servidor web embebido, validación, tareas asíncronas
Spring JDBC (JdbcTemplate)	gestionada por BOM	Backend	Acceso a datos con SQL directo y RowMapper manuales
Spring Security Crypto	gestionada por BOM	Backend	Codificación de contraseñas con BCrypt
PostgreSQL	16 (15 en tests)	Base de datos	Motor relacional
postgresql (driver)	gestionada por BOM	Backend	Driver JDBC para PostgreSQL
Flyway	gestionada por BOM	Backend	Migraciones de esquema versionadas (V1-V28)
springdoc-openapi	2.8.5	Backend	Swagger UI y especificación OpenAPI 3
Apache Maven	3.8+	Backend + cliente	Construcción y gestión de dependencias
JavaFX	21.0.2	Cliente	Interfaz gráfica de escritorio
java.net.http.HttpClient	nativo Java 21	Cliente	Comunicación HTTP con el backend
Jackson Databind + JSR-310	2.17.0	Cliente	Serialización JSON, incluido el manejo de fechas java.time
jpackage / maven-shade	plugin 1.6.5	Cliente	Fat JAR e instaladores nativos (.exe, .deb, .dmg)
React	19	Frontend	Framework de interfaz web basado en componentes
TypeScript	5	Frontend	Tipado estático sobre JavaScript
Vite	6	Frontend	Bundler y servidor de desarrollo

React Router	7	Frontend	Enrutado client-side de la SPA
Tailwind CSS	3	Frontend	Framework CSS utilitario
Recharts	2	Frontend	Gráficos de tendencias y macros
html5-qrcode	2	Frontend	Lectura de códigos de barras con la cámara
Framer Motion	11	Frontend	Animaciones declarativas en React
Vitest + Testing Library	2 / 16	Frontend	Tests de componentes y contexto
JUnit 5 + Mockito + AssertJ	gestionada por BOM	Backend + cliente	Tests unitarios y aserciones
Testcontainers	gestionada por BOM	Backend	PostgreSQL real en Docker para tests de integración
OpenRouter API	—	Backend	Proxy de LLM para las funciones de IA
Open Food Facts API	—	Backend	Datos de alimentos por código de barras
Wger API	—	Backend	Catálogo externo de ejercicios
Git + GitHub	—	Repositorio	Control de versiones
GitHub Actions	—	CI/CD	Compilación, tests e instaladores en cada push y release
Docker	—	Infraestructura	Imagen del backend y PostgreSQL en Testcontainers
Render	—	Despliegue	Hosting del backend y base de datos (plan gratuito)
Vercel	—	Despliegue	Hosting del frontend (plan gratuito)

3.2 Backend

3.2.1 Java 21

El proyecto compila con Java 21, la versión LTS más reciente. Java 21 aporta bloques de texto para las consultas SQL multilínea en los repositorios, la API `java.time` para el manejo de fechas en el dominio y mejoras en la gestión de hilos que, aunque no se utilizan directamente aquí, hacen que Java 21 sea la base más sólida disponible.

3.2.2 Spring Boot 3.5.0

Spring Boot gestiona el ciclo de vida del servidor, la inyección de dependencias, la validación de peticiones con Bean Validation y las conexiones JDBC. Se usan los starters `spring-boot-starter-web` para el servidor REST, `spring-boot-starter-validation` para validación declarativa y `spring-`

`boot-starter-jdbc` para `JdbcTemplate` con pool `HikariCP`. Las funciones de IA que tardan más de lo aceptable para una petición síncrona (plan semanal y detective) se ejecutan con `@Async` sobre un executor configurado en `AsyncConfig`.

3.2.3 Spring JDBC y JdbcTemplate

El acceso a datos se implementa íntegramente con `JdbcTemplate` y `RowMapper` manuales. No se usa JPA, Spring Data ni Hibernate. Cada módulo declara una interfaz `XxxRepository` y una implementación `JdbcXxxRepository` con SQL explícito. La justificación de esta elección frente a JPA se desarrolla en §5.3.1.

3.2.4 PostgreSQL y Flyway

PostgreSQL almacena el estado del sistema. El esquema no se define manualmente: Flyway aplica las veintiocho migraciones al arrancar el backend. Para evitar fallos cuando una migración ya aplicada cambia de checksum entre entornos, una estrategia personalizada ejecuta `repair()` antes de `migrate()`. La última migración (V28) añade un trigger que limpia las sesiones caducadas, de modo que la tabla no acumula tokens muertos.

3.2.5 Integración con APIs externas

OpenRouter actúa como proxy entre el backend y los modelos de lenguaje. El backend construye el contexto nutricional del usuario (resumen diario, perfil, historial reciente) y lo envía a OpenRouter; la clave y el modelo pueden ser globales o, si el usuario lo configura, propios y guardados en `usuario_ia_config`. Open Food Facts proporciona datos nutricionales por código de barras EAN. Wger aporta un catálogo de ejercicios que se cachea en memoria durante seis horas para no repetir descargas. Las tres integraciones degradan de forma controlada: si el servicio externo no responde, el endpoint devuelve un error claro o una lista vacía en lugar de propagar la excepción.

3.3 Frontend React

3.3.1 React 19 + TypeScript

La interfaz web es una SPA construida con React 19 y TypeScript. React gestiona el estado de la UI mediante hooks; TypeScript añade tipado estático que elimina una categoría entera de errores en tiempo de compilación. La interfaz tiene catorce páginas: Dashboard, Alimentos, Comidas, Ejercicios, Escáner, Hidratación, Detective, Opciones IA, Perfil, Plan semanal, Retos, Tendencias, Login y Registro.

3.3.2 Vite, Tailwind y gráficos

Vite actúa como bundler y servidor de desarrollo, con un arranque en caliente inferior a un segundo. Tailwind CSS gestiona los estilos mediante clases utilitarias aplicadas en JSX, sin archivos CSS por componente. Recharts dibuja los gráficos de peso, NutriScore y

macronutrientes; html5-qrcode usa la cámara para leer códigos de barras; Framer Motion añade transiciones entre páginas.

3.4 Cliente JavaFX

El cliente JavaFX cubre el núcleo de uso diario: acceso, alimentos, comidas, ejercicios, hidratación, perfil, historial de peso, resumen diario con evaluación IA, gamificación y retos. Consume el mismo backend mediante `java.net.http.HttpClient` y Jackson, con un cliente base común que centraliza el timeout, el token Bearer y el manejo de fechas. Las operaciones de red se ejecutan en hilos de fondo con `javafx.concurrent.Task`. Se empaqueta como fat JAR con maven-shade y como instalador nativo con jpackage para Windows, Linux y macOS.

3.5 Despliegue

El backend se despliega en Render como imagen Docker con build automático desde la rama `main`, junto a una base de datos PostgreSQL gestionada. El frontend se despliega en Vercel con el mismo disparador. Ambos servicios sirven HTTPS sin configuración adicional. El backend en el plan gratuito de Render se suspende tras quince minutos de inactividad; la primera petición tras ese período tiene una latencia de arranque de treinta a sesenta segundos.

4. Análisis de requisitos

4.1 Introducción

Los requisitos emergieron de forma iterativa a medida que los módulos se definían e implementaban. El objetivo de documentarlos aquí es hacer explícito qué hace el sistema, qué restricciones de calidad cumple y qué queda fuera del alcance actual.

4.2 Requisitos funcionales

Módulo de autenticación

Código	Requisito
RF-01	El sistema permite registrar un nuevo usuario con nombre, email y contraseña. El registro abre sesión automáticamente.
RF-02	Un usuario registrado puede autenticarse con su email y contraseña. Si las credenciales son correctas, el sistema genera un token de sesión.
RF-03	Un usuario puede cerrar su sesión. La invalidación del token es inmediata y del lado del servidor.
RF-04	El sistema normaliza el email antes de cualquier búsqueda: aplica <code>trim()</code> y <code>toLowerCase()</code> .
RF-05	No pueden existir dos usuarios con el mismo email. El intento de registro con email duplicado se rechaza antes de persistir ningún dato.

Módulo de alimentos

Código	Requisito
RF-06	El sistema mantiene un catálogo de alimentos compartido. Cualquier usuario puede consultar el catálogo completo o filtrar por nombre.
RF-07	La búsqueda por nombre es parcial e insensible a mayúsculas.
RF-08	Se puede crear un alimento especificando nombre, porción de referencia, kcal/100g, proteínas/100g, grasas/100g e hidratos/100g.
RF-09	Un alimento existente puede actualizarse o eliminarse. No se puede eliminar un alimento referenciado en algún ítem de comida.
RF-10	El escáner busca un alimento en Open Food Facts por código de barras EAN. De forma complementaria, una foto del producto puede analizarse con un modelo de visión para proponer sus macros.

Módulo de comidas

Código	Requisito
RF-11	Un usuario puede registrar una comida para una fecha concreta con un tipo de toma (DESAYUNO, ALMUERZO, CENA, etc.).
RF-12	A una comida se le pueden añadir ítems. Cada ítem asocia un alimento del catálogo con una cantidad en gramos.
RF-13	Al consultar los ítems de una comida, el sistema calcula y devuelve los valores nutricionales ponderados por gramos. El cálculo se realiza en base de datos.
RF-14	Una comida puede eliminarse. Al hacerlo, todos sus ítems se eliminan en cascada.

Módulo de resumen diario y evaluación IA

Código	Requisito
RF-15	El sistema calcula y devuelve el total de kcal, proteínas, grasas e hidratos consumidos por un usuario en una fecha.
RF-16	Si el usuario no tiene comidas en la fecha consultada, el resumen devuelve ceros. No se produce error.
RF-17	El sistema genera una evaluación nutricional del resumen diario mediante un modelo de lenguaje, enriquecida con el contexto de los últimos siete días.

Módulo de perfil

Código	Requisito
RF-18	Un usuario puede configurar su perfil biométrico: sexo, fecha de nacimiento, altura, peso actual, nivel de actividad y, opcionalmente, peso objetivo.
RF-19	El sistema calcula la TMB y el TDEE a partir del perfil, usando la fórmula de Mifflin-St Jeor. El cálculo se realiza en el backend.
RF-20	Un usuario puede actualizar su perfil en cualquier momento. El sistema recalcula TMB y TDEE de forma automática.

Módulo de ejercicios

Código	Requisito
RF-21	El sistema mantiene un catálogo de ejercicios con su valor MET y su tipo (aeróbico o anaeróbico). Cualquier usuario autenticado puede consultarlo.
RF-22	Un usuario puede registrar una sesión aeróbica indicando fecha y duración, o una sesión anaeróbica indicando series e intensidad.

RF-23	El sistema calcula las calorías quemadas. En aeróbico: $\text{MET} \times \text{peso_kg} \times (\text{duración_min} / 60)$. En anaeróbico: a partir del MET, el número de series y un factor de intensidad. El resultado se redondea a dos decimales.
RF-24	El peso utilizado en el cálculo se toma del perfil en el momento del registro y se persiste junto al resultado.

Módulos adicionales

Código	Módulo	Requisito
RF-25	Hidratación	El usuario puede registrar el agua consumida por día. El sistema devuelve el total y el porcentaje frente al objetivo de dos litros.
RF-26	Peso historial	El usuario puede registrar su peso en distintas fechas. El sistema devuelve el historial ordenado cronológicamente.
RF-27	Tendencias	El sistema calcula series de los últimos noventa días: peso, NutriScore, macronutrientes (por semana) y ejercicio.
RF-28	Plan semanal	El sistema genera, de forma asíncrona, un plan de comidas para la semana mediante IA, adaptado al perfil y objetivos del usuario.
RF-29	Detective	El sistema analiza el historial reciente (7 a 90 días), calcula estadísticas de cumplimiento y emite hallazgos, ampliados con una conclusión de IA.
RF-30	Retos	El sistema mantiene un catálogo de retos de fitness. El usuario puede inscribirse y registrar progreso.
RF-31	Gamificación	El sistema calcula una racha de días activos y una puntuación nutricional diaria (NutriScore) sobre proteína, balance, ejercicio y variedad.
RF-32	Config. IA	El usuario puede configurar el modelo, la clave y, opcionalmente, una URL de proxy de IA. La clave se devuelve enmascarada y la URL se valida.

4.3 Requisitos no funcionales

Código	Requisito
RNF-01	Todos los campos obligatorios se validan con Bean Validation antes de que la petición llegue al servicio. Las restricciones incumplidas devuelven HTTP 400.
RNF-02	Los valores numéricos de alimentos no pueden ser negativos. La altura del perfil debe estar entre 100 y 250 cm; el peso, por encima de 20 kg.
RNF-03	Todos los errores del backend tienen la misma estructura JSON: <code>timestamp</code> , <code>status</code> , <code>error</code> , <code>message</code> y <code>path</code> . Los detalles internos de la excepción no se exponen al cliente.
RNF-04	Los mensajes de error de login son idénticos para email no encontrado y contraseña incorrecta, para impedir enumeración de usuarios.

RNF-05	El esquema de la base de datos está gestionado exclusivamente por Flyway. El código Java nunca crea ni modifica tablas.
RNF-06	Las contraseñas se almacenan exclusivamente como hash BCrypt, con un mínimo de ocho caracteres. El texto en claro nunca se persiste ni se registra.
RNF-07	Los tokens de sesión son UUIDs v4 con expiración de siete días, enviados como cookie <code>HttpOnly; Secure; SameSite=None</code> . Al hacer logout, la fila se elimina.
RNF-08	Cada endpoint sobre datos de usuario comprueba que el identificador solicitado coincide con el del token. Un acceso a datos ajenos devuelve HTTP 401.
RNF-09	El login y el registro están limitados a 10 peticiones por minuto y dirección IP; los endpoints de IA, a 5 por minuto y usuario.
RNF-10	El backend añade cabeceras de seguridad en todas las respuestas: HSTS, CSP, <code>X-Frame-Options</code> , <code>X-Content-Type-Options</code> , <code>Referrer-Policy</code> y <code>Permissions-Policy</code> .
RNF-11	Los tests unitarios se ejecutan con Mockito, sin base de datos ni contexto de Spring. Los tests de integración usan Testcontainers con un PostgreSQL real.
RNF-12	El frontend y el backend están desplegados en HTTPS. El backend solo acepta peticiones desde los orígenes configurados en CORS (Vercel y <code>localhost:5173</code>).

4.4 Exclusiones conocidas

El sistema no implementa diferenciación de roles entre usuarios: todos los autenticados tienen el mismo nivel de acceso. El catálogo de alimentos y ejercicios es compartido, sin un subconjunto propio por usuario. El cliente de escritorio no cubre los módulos de IA más recientes (detective, plan semanal, tendencias y configuración de IA), que sí están en la web. La función de lista de la compra, presente en un MVP anterior, se retiró durante el desarrollo y su tabla se eliminó en la migración V27. Estas limitaciones son consecuencia del alcance definido, no de errores de diseño.

5. Diseño del sistema

5.1 Arquitectura

NutriFit tiene tres capas con separación física:

Capa	Tecnología	Responsabilidad principal
Presentación web	React 19 + TypeScript (Vercel)	SPA, enrutado client-side, llamadas a la API
Presentación escritorio	JavaFX 21	Interfaz gráfica de escritorio, llamadas HTTP
Lógica de aplicación	Spring Boot 3.5 (Render)	API REST, reglas de negocio, acceso a datos
Persistencia	PostgreSQL (base de datos <code>nutrifit</code>)	Almacenamiento relacional

El backend no sabe nada del cliente; cualquier consumidor que respete el contrato HTTP obtiene el mismo comportamiento. Ambos clientes son módulos independientes que pueden construirse y probarse por separado.

5.1.1 Capa de presentación: frontend React

El frontend es una SPA servida desde Vercel. La navegación es client-side mediante React Router; las páginas no se recargan desde el servidor. El enrutado protegido se implementa con un componente `ProtectedRoute` que verifica la sesión en el contexto de autenticación antes de renderizar la página. Las peticiones a la API se centralizan en funciones del directorio `src/api/`, una por módulo, sobre una instancia de Axios con `withCredentials` activado para enviar la cookie de sesión.

5.1.2 Capa de presentación: cliente JavaFX

El cliente JavaFX cubre el núcleo de uso diario. Dentro del cliente, los componentes se organizan en tres grupos: pantallas FXML con sus controladores, clases `ApiClient` por módulo que encapsulan las llamadas HTTP, y un `SessionManager` estático que mantiene en memoria el token y los datos de sesión. El token no se persiste en disco; desaparece al cerrar la aplicación. Diez pantallas están implementadas (acceso, alimentos, comidas, ejercicios, hidratación, perfil, peso, resumen, gamificación y retos); los módulos de IA más recientes quedan, por ahora, solo en la web.

5.1.3 Capa de lógica de aplicación: el backend Spring Boot

El backend sigue la misma estructura en capas verticales para todos sus módulos:

Los controladores reciben la petición HTTP, comprueban la propiedad del recurso, validan con `@Valid` y delegan en el servicio. Los servicios contienen las reglas de negocio. Los repositorios encapsulan el acceso JDBC. Un `GlobalExceptionHandler` centralizado convierte cualquier excepción en una respuesta JSON con estructura uniforme.

5.1.4 Capa de persistencia: PostgreSQL y Flyway

PostgreSQL almacena el estado del sistema. El esquema se construye mediante las veintiocho migraciones de Flyway en `backend/src/main/resources/db/migration/`. Si el esquema ya está al día cuando el backend arranca, Flyway lo valida y no ejecuta nada.

5.2 Diseño de la base de datos

5.2.1 Migraciones Flyway

Migración	Contenido
V1, V2	Tablas <code>usuarios</code> y <code>alimentos</code>
V3	Tabla <code>sesiones</code>
V4	Seed inicial del catálogo de alimentos
V5	Tablas <code>comidas</code> y <code>comida_alimentos</code>
V6, V7	Tabla <code>ejercicios</code> y seed inicial
V8, V9	Función de resumen diario y su actualización
V10	Índices de rendimiento
V11	Seed expandido de alimentos
V12	Tabla <code>peso_historial</code>
V13	Índices y restricciones CHECK adicionales
V14	Tabla <code>agua_registro</code>
V15	Tabla <code>plan_semanal</code>
V16	Tablas <code>retos</code> y <code>usuario_retos</code> con siete retos seed
V17	Tabla <code>lista_compra</code> (retirada después en V27)
V18	Columna <code>tipo</code> (aeróbico/anaeróbico) en ejercicios
V19	Seed completo del catálogo de ejercicios (más de 200)
V20	Campos de intensidad y número de series en registros

V21	Relajación de la restricción de duración mínima
V22	Tabla <code>usuario_ia_config</code>
V23	Tabla <code>alimentos_ocultos</code> (alimentos ocultos por usuario)
V24	Deduplicación e índice único insensible a mayúsculas en nombres de alimento
V25	Columnas de estado y error en <code>plan_semanal</code> (generación asíncrona)
V26	Tabla <code>detective_analisis</code>
V27	Eliminación de la tabla <code>lista_compra</code>
V28	Trigger y función de limpieza de sesiones caducadas

5.2.2 Principales relaciones

Entidad origen	Cardinalidad	Entidad destino	Al borrar origen
usuarios	1 — 0..*	sesiones	ON DELETE CASCADE
usuarios	1 — 0..*	comidas, peso_historial, agua_registro, plan_semanal, usuario_retos	ON DELETE CASCADE
usuarios	1 — 1	usuario_ia_config, detective_analisis	ON DELETE CASCADE
usuarios	N — M	alimentos (vía alimentos_ocultos)	ON DELETE CASCADE
comidas	1 — 0..*	comida_alimentos	ON DELETE CASCADE
alimentos	1 — 0..*	comida_alimentos	ON DELETE RESTRICT
ejercicios	1 — 0..*	ejercicios_registro	ON DELETE RESTRICT

La regla `ON DELETE RESTRICT` en alimentos y ejercicios impide eliminar un elemento del catálogo si existe algún registro que lo referencia, protegiendo la integridad del historial.

5.3 Decisiones de diseño

5.3.1 Acceso a datos con Spring JDBC en lugar de JPA/Hibernate

El acceso a la base de datos se implementa con `JdbcTemplate` y `RowMapper` manuales. El argumento principal es de trazabilidad: con `JdbcTemplate`, cada consulta es SQL visible en una línea concreta del código. En una defensa, responder "¿qué SQL genera esta línea?" es inmediato

con JDBC y requiere depuración con JPA. Como ventaja adicional, Flyway gestiona el esquema sin interferencias; no hay que coordinar `ddl-auto`.

5.3.2 Autenticación con token opaco en lugar de JWT

La autenticación usa un UUID aleatorio almacenado en la tabla `sesiones`, con fecha de expiración y borrado explícito al hacer logout. JWT no resuelve bien el logout: un token firmado sigue siendo válido hasta que expira, aunque el usuario lo haya invalidado. Forzar la invalidación inmediata con JWT requeriría una lista negra en base de datos, que es la misma infraestructura que ya se tiene, pero con más complejidad.

5.3.3 Cálculo del resumen diario con SQL

El resumen diario se calcula con una única consulta con LEFT JOIN, SUM, ROUND y COALESCE. La alternativa descartada era traer todas las comidas e ítems al servicio Java y agregar en memoria. Eso habría requerido varias consultas y lógica de agregación en la capa de aplicación, desaprovechando el motor de base de datos.

5.3.4 Cálculo de TMB/TDEE en el backend

La fórmula de Mifflin-St Jeor reside en el servicio de backend. Los valores calculados se devuelven en la respuesta. Esto tiene dos consecuencias prácticas: la lógica está en un único lugar y puede verificarse con tests unitarios independientes del cliente; y si el cliente cambia de tecnología, la fórmula no se duplica.

5.3.5 Configuración de IA por usuario

La clave y el modelo de OpenRouter se pueden almacenar por usuario en `usuario_ia_config`, no solo en variables de entorno globales. Así cada usuario puede usar su propia integración sin que el administrador gestione una clave compartida. El inconveniente es que el usuario necesita una cuenta en OpenRouter, lo que añade fricción al onboarding. La estrategia de reintento prueba primero la configuración propia y, si falla, recurre a los modelos por defecto.

5.3.6 PostgreSQL en lugar de MariaDB

El proyecto evolucionó de MariaDB a PostgreSQL. Los motivos son prácticos: Render ofrece PostgreSQL gestionado en su plan gratuito, pero no MariaDB. Además, PostgreSQL tiene mejor soporte para funciones y triggers en PL/pgSQL y para los tipos usados en agregaciones temporales.

5.3.7 Seguridad como capa transversal

En lugar de tratar la seguridad como un añadido final, el sistema la reparte en varias defensas independientes: un interceptor valida el token y expone el identificador autenticado, cada controlador comprueba la propiedad del recurso, dos limitadores de tasa protegen el login y los endpoints de IA, un filtro añade cabeceras de seguridad y la configuración de IA valida la URL de

proxy. Cada una cubre un fallo distinto, de modo que un descuido en una no deja el sistema abierto.

6. Implementación

6.1 Módulo de autenticación

El módulo expone tres endpoints bajo `/api/auth`: `POST /register` (201), `POST /login` (200) y `POST /logout` (204). El registro aplica `trim()` y `toLowerCase()` al email, comprueba unicidad, hashea la contraseña con BCrypt, persiste el usuario, genera un UUID v4 como token y devuelve el `AuthResponse` junto a una cookie `nf_session`. El registro abre sesión automáticamente.

En el login, si el email no existe o la contraseña no coincide, el servicio lanza la misma excepción con el mismo mensaje, lo que impide deducir si un email está registrado. El acceso de fuerza bruta se mitiga con un limitador de 10 intentos por minuto y dirección IP. En cada petición posterior, un interceptor lee el token de la cookie (o, como alternativa, de la cabecera `Authorization`), verifica que la sesión existe y no ha caducado, y deja el identificador del usuario disponible para que el resto de controladores comprueben la propiedad del recurso.

6.2 Módulo de alimentos y escáner

El módulo mantiene un catálogo compartido con CRUD completo. Los valores nutricionales se modelan como `BigDecimal` para preservar la precisión decimal. La búsqueda por nombre usa `LIKE` con comodines, insensible a mayúsculas. Un índice único insensible a mayúsculas (V24) evita duplicados como "Arroz" y "arroz".

El escáner por código de barras (`GET /api/escaner/{barcode}`) llama a Open Food Facts, extrae los campos nutricionales y los devuelve en el mismo formato que un alimento del catálogo. Cuando el valor calórico no viene directo, se deduce a partir de la energía en kilojulios o, en último término, de los macros. Si el código no existe, el endpoint devuelve 404. De forma complementaria, `POST /api/alimentos/escanear-foto` envía una imagen (validada en tipo y tamaño) a un modelo de visión que propone los macros del producto.

6.3 Módulo de comidas

El módulo distingue dos entidades: `Comida` (una toma del día, identificada por usuario, fecha y tipo) y `ComidaAlimento` (la relación entre una comida y un alimento, con los gramos consumidos). Al añadir un ítem, el servicio verifica primero que tanto la comida como el alimento existen, aplicando el patrón fail-fast.

6.4 Módulo de resumen diario y evaluación IA

El resumen diario se calcula con una única consulta que cruza `comidas`, `comida_alimentos` y `alimentos` con dos `LEFT JOIN`. La fórmula por ítem es $(\text{valor_por_100g} \times \text{gramos}) / 100$.

`COALESCE` convierte en ceros los nulos que produce `SUM` sobre un conjunto vacío.

La evaluación IA toma el resumen del día, el perfil y un contexto de siete días (medias de kcal y proteínas, días con ejercicio y balance medio), construye un prompt estructurado y lo envía a OpenRouter. Antes de llamar al modelo, un limitador de 5 peticiones por minuto y usuario protege la clave de API frente a abuso. La respuesta del modelo se devuelve al cliente.

6.5 Módulo de perfil y cálculo TMB/TDEE

Los datos biométricos se almacenan en la tabla `usuarios`. El módulo expone `GET /api/perfil/{id}` y `PUT /api/perfil/{id}`. La fórmula de Mifflin-St Jeor se implementa en `PerfilServiceImpl`:

```
private double calcularTmb(Perfil perfil) {
    int edad = Period.between(perfil.getFechaNacimiento(), LocalDate.now()).getYears();
    double base = 10 * perfil.getPesoKgActual()
        + 6.25 * perfil.getAlturaCm()
        - 5 * edad;
    return perfil.getSexo() == Sexo.H ? base + 5 : base - 161;
}
```

La edad se calcula con `Period.between` para garantizar el resultado correcto independientemente del día de ejecución. El TDEE multiplica la TMB por el factor del nivel de actividad.

6.6 Módulo de ejercicios

El módulo gestiona el catálogo de ejercicios y los registros de sesiones. En ejercicio aeróbico las calorías se calculan con $MET \times peso_kg \times (duración_min / 60.0)$; en anaeróbico, a partir del MET, el número de series y un factor de intensidad (baja, media o alta), redondeando a dos decimales con `HALF_UP`. El peso se toma del perfil en el momento del registro y se persiste junto al resultado, para que el historial no cambie si el usuario actualiza su peso después.

6.7 Módulos de gamificación, detective y tendencias

La gamificación calcula dos cosas. La racha cuenta días consecutivos hacia atrás con al menos una comida registrada. El NutriScore puntúa el día de 0 a 100 sobre cuatro criterios de 25 puntos (proteína suficiente, balance dentro de margen, algo de ejercicio y variedad de alimentos) y lo traduce a una letra de la A a la F.

El detective analiza entre 7 y 90 días: calcula la tasa de registro, el déficit real medio, el cumplimiento de proteína y el patrón por día de la semana, y de ahí emite hallazgos (críticos, advertencias o positivos). El análisis se lanza de forma asíncrona y, una vez calculadas las estadísticas, pide a la IA una conclusión en lenguaje natural. El módulo de tendencias devuelve series de peso, NutriScore, macronutrientes (agrupados por semana ISO) y ejercicio para alimentar los gráficos de la web.

6.8 Módulos de hidratación, peso, plan semanal y retos

Hidratación, peso historial y retos siguen la estructura Controller → Service → Repository. La hidratación compara el total del día con un objetivo de 2000 ml. El plan semanal se genera con IA de forma asíncrona: el registro se crea en estado GENERANDO y un proceso en segundo plano lo completa o lo marca con error, de modo que la petición no se queda bloqueada. Los módulos con IA comparten la misma estrategia de llamada a OpenRouter y propagan los errores de cuota o autenticación como excepciones controladas.

6.9 Filtros y seguridad transversal

Dos filtros se aplican a todas las respuestas. `RequestLoggingFilter` registra método, ruta, estado y duración. `SecurityHeadersFilter` añade las cabeceras de seguridad (HSTS, CSP, `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy` y `Permissions-Policy`). La validación de la URL de proxy de IA vive en el servicio de configuración, que rechaza cualquier host privado o de loopback.

6.10 Frontend React

El frontend organiza las llamadas en funciones del directorio `src/api/` sobre una instancia de Axios con `withCredentials` y un timeout amplio para tolerar el arranque en frío de Render y las llamadas a IA. El estado de autenticación se gestiona con un contexto (`AuthContext`) que expone el usuario, persiste sus datos no sensibles en `localStorage` y reacciona a las respuestas 401. Los gráficos incluyen tendencias de peso, distribución de macros, mapa de calor de ejercicios y puntuación nutricional.

6.11 Cliente JavaFX

El cliente organiza la comunicación en clases `ApiClient` por módulo sobre un cliente base común. Cada llamada construye la petición con `HttpRequest.newBuilder()`, serializa el cuerpo con Jackson, comprueba el código de estado y, si no es 2xx, lanza una excepción con el mensaje del cuerpo de error. El cliente base registra el módulo de fechas de Jackson para deserializar correctamente los `LocalDate` que devuelve el backend. Todas las operaciones de red se envuelven en `javafx.concurrent.Task`.

6.12 Integración continua y despliegue

GitHub Actions compila el backend y el cliente con Java 21 y ejecuta las suites de tests en cada push, de modo que ningún commit deja el proyecto sin compilar o con tests rotos. Un segundo flujo, disparado al crear una etiqueta de versión, construye la imagen Docker del backend y genera los instaladores nativos del cliente en una matriz de Windows, Linux y macOS con `jpackage`, y publica una release de GitHub con los tres instaladores. El frontend se despliega en

Vercel y el backend en Render en cada push a `main`; las migraciones de Flyway se aplican en el primer arranque del contenedor.

7. Pruebas

7.1 Estrategia de pruebas

La estrategia combina cuatro enfoques. Los tests unitarios del backend verifican la capa de servicio con Mockito, sin base de datos ni contexto de Spring, y se ejecutan en pocos segundos. Los tests de integración levantan un PostgreSQL real con Testcontainers y prueban los endpoints de extremo a extremo. El cliente JavaFX tiene tests unitarios sobre sus modelos, su cliente base y varios controladores. El frontend tiene tests de componentes con Vitest. En total, 199 tests automatizados.

La cobertura automatizada se concentra en la lógica de negocio, que es donde residen las reglas de dominio, los casos de error y los cálculos nutricionales. Los endpoints se cubren además con los tests de integración y con peticiones manuales documentadas.

7.2 Tests unitarios del backend

Usan JUnit 5, Mockito y AssertJ. Ningún test de esta categoría levanta el contexto de Spring: todos usan `@ExtendWith(MockitoExtension.class)`.

Clase de test	Tests	Operaciones cubiertas
AlimentoServiceImplTest	18	CRUD, búsqueda, análisis de foto con IA
ComidaServiceImplTest	21	comidas, ítems, cálculo de macros, control de acceso
RegistroEjercicioServiceImplTest	14	registro, cálculo MET aeróbico y anaeróbico
ResumenDiarioServiceImplTest	14	balance, TDEE, estado, fecha objetivo
GamificacionServiceTest	14	NutriScore (A-F), racha, criterios
AuthServiceImplTest	9	register, login, logout, enumeración
EjercicioServiceImplTest	9	catálogo, filtros, integración Wger
HidratacionServiceImplTest	8	registro, total diario, borrado
PesoHistorialServiceImplTest	8	upsert, historial, borrado
PerfilServiceImplTest	5	getPerfil, updatePerfil, TMB/TDEE
Subtotal unitarios	120	

7.3 Tests de integración del backend (Testcontainers)

Una clase base levanta un contenedor PostgreSQL 15 y registra su URL, usuario y contraseña con `@DynamicPropertySource`. Sobre esa base de datos real se ejecutan las migraciones de Flyway y se prueban los endpoints con peticiones HTTP a un puerto aleatorio. Así se validan migraciones, restricciones, claves foráneas e índices reales, no un H2 que se comporta distinto.

Clase de test	Tests	Qué prueba
AuthControllerIT	5	registro, login, logout, rutas protegidas
ComidaControllerIT	3	CRUD de comidas e ítems, propiedad
GamificacionControllerIT	3	NutriScore y racha vía endpoint
PerfilControllerIT	3	obtener y actualizar perfil, recálculo
HidratacionControllerIT	2	registro y consulta diaria
BaseIntegrationTest	1	arranque del contexto e infraestructura
Subtotal integración	17	

Total backend: 120 unitarios + 17 de integración = 137 tests.

7.4 Tests del cliente JavaFX

Clase de test	Tests	Qué prueba
SessionManagerTest	8	gestión de sesión en memoria
AlimentoFxTest	5	mapeo de DTOs de alimento
BaseApiClientTest	5	configuración del cliente HTTP y errores
PerfilDtoTest	3	deserialización del perfil con TMB/TDEE
ResumenDiarioDtoTest	3	resumen diario con ceros y valores reales
PesoHistorialControllerTest	3	carga y registro de peso
GamificacionControllerTest	2	carga de NutriScore y racha
RetosControllerTest	2	carga del listado de retos
Total JavaFX	31	

7.5 Tests del frontend (Vitest)

Suite	Tests	Qué prueba
Alimentos.test.tsx	6	listado, búsqueda, alta de alimento

AuthContext.test.tsx	4	login/logout y persistencia de sesión
Comidas.test.tsx	4	registro de comidas y cálculo de macros
Dashboard.test.tsx	4	resumen diario, anillo de macros
ErrorBoundary.test.tsx	4	captura de errores y UI de respaldo
Register.test.tsx	4	formulario y validación de registro
Login.test.tsx	3	formulario, errores y envío
ProtectedRoute.test.tsx	2	redirección sin sesión
Total frontend	31	

7.6 Resumen y aspectos destacados

Módulo	Tests	Framework
Backend (unit + integración)	137	JUnit 5 + Mockito + AssertJ + Testcontainers
Cliente JavaFX	31	JUnit 5 + Mockito + AssertJ
Frontend React	31	Vitest + Testing Library
Total	199	

Algunos tests merecen mención. Los de `AuthServiceImplTest` comprueban que el registro con email duplicado lanza la excepción antes de invocar `save`, que la sesión creada tiene token y expiración futura (verificado con `ArgumentCaptor`) y que el mensaje de error es idéntico para email inexistente y contraseña incorrecta. Los de `PerfilServiceImplTest` verifican la fórmula de Mifflin-St Jeor con valores calculados a mano para cada sexo, usando una fecha de nacimiento relativa para no depender del día de ejecución. Los de `RegistroEjercicioServiceImplTest` comprueban el redondeo: MET 5,0, 70 kg y 20 min producen 116,67 kcal, no 116,66.

7.7 Pruebas manuales de la API

Todos los endpoints se han verificado con peticiones HTTP reales. El directorio `docs/api/` contiene archivos `.http` compatibles con IntelliJ IDEA y con la extensión REST Client de VS Code, organizados por módulo. Swagger UI, en `/swagger-ui.html`, complementa las pruebas permitiendo explorar y ejecutar peticiones desde el navegador.

8. Seguridad

8.1 Gestión de contraseñas

Las contraseñas nunca se almacenan en texto plano. Al registrarse, `AuthServiceImpl` delega el hash en `PasswordService`, que usa `BCryptPasswordEncoder` de Spring Security. BCrypt incorpora una sal aleatoria, de modo que dos hashes del mismo texto producen valores distintos, lo que neutraliza los ataques de diccionario precomputados. La contraseña debe tener al menos ocho caracteres, validado en el DTO de registro. Durante el login, `matches()` compara la contraseña recibida con el hash; si falla, el servicio devuelve el mismo mensaje genérico que cuando el email no existe.

8.2 Autenticación basada en token opaco

NutriFit usa un token opaco, un UUID generado con `UUID.randomUUID()`, almacenado en la tabla `sesiones` con restricción `UNIQUE`. En cada petición autenticada, `AuthInterceptor` lo extrae de la cookie `nf_session` (o de la cabecera `Authorization` como alternativa para el cliente de escritorio), comprueba que la sesión existe y que no ha caducado, y deja el identificador del usuario en la petición. Si algo falla, lanza `UnauthorizedException`, que se traduce en HTTP 401. Al hacer logout, la fila se elimina y cualquier petición posterior con ese token recibe 401. Además, el trigger de la migración V28 borra periódicamente las sesiones caducadas.

En la web, el token viaja como cookie `HttpOnly; Secure; SameSite=None`: `HttpOnly` impide que JavaScript lo lea (mitiga el robo por XSS), `Secure` obliga a HTTPS y `SameSite=None` es necesario porque el frontend (Vercel) y el backend (Render) están en dominios distintos. El cliente JavaFX guarda el token solo en memoria.

8.3 Control de propiedad de los recursos

El identificador autenticado que deja el interceptor permite que cada controlador compruebe que el usuario solo accede a sus propios datos. Antes de devolver o modificar comidas, perfil, registros o cualquier recurso ligado a una cuenta, se compara el `usuarioId` de la petición con el del token; si no coinciden, la respuesta es 401. Como segunda barrera, las operaciones de borrado filtran también por `usuario_id` en el `WHERE`, de modo que un identificador manipulado no afecta a filas ajenas.

8.4 Cabeceras de seguridad y CORS

Un filtro añade en todas las respuestas un conjunto de cabeceras: `Strict-Transport-Security` (HSTS de un año), `Content-Security-Policy` restrictiva, `X-Frame-Options: DENY` contra clickjacking, `X-Content-Type-Options: nosniff`, `Referrer-Policy` y `Permissions-Policy` que

desactiva cámara, micrófono y geolocalización. El backend solo acepta peticiones desde los orígenes configurados en CORS: el dominio de Vercel y `localhost:5173`. El cliente JavaFX habla directamente con el backend y no está sujeto a CORS.

8.5 Limitación de tasa

Dos limitadores en memoria, con ventana deslizante, protegen los puntos sensibles. El de login y registro permite 10 intentos por minuto y dirección IP, lo que frena la fuerza bruta sobre credenciales. El de IA permite 5 peticiones por minuto y usuario en evaluación, plan semanal, detective y análisis de foto, lo que evita que un usuario agote la clave de API contra los servicios externos. Al superar el límite, la respuesta es 429 con cabecera `Retry-After`.

8.6 Validación de entrada y protección frente a SSRF

Todos los DTO se validan con Bean Validation antes de llegar al servicio. El análisis de foto, por ejemplo, limita el tamaño de la imagen y restringe el tipo MIME a formatos de imagen conocidos, lo que evita un abuso de memoria. La configuración de IA permite definir una URL de proxy propia, que es un vector clásico de SSRF; por eso se valida contra una lista de hosts privados y de loopback (localhost, 127.x, 10.x, 192.168.x, 169.254.x y similares) antes de usarla. La clave de API del usuario se devuelve siempre enmascarada, mostrando solo sus últimos cuatro caracteres.

8.7 Privacidad y datos de salud

NutriFit almacena datos de salud: perfil biométrico, historial de peso, comidas, ejercicios e hidratación. Las medidas son hash BCrypt para las contraseñas, borrado en cascada de todos los datos del usuario al eliminar la cuenta, y expiración de sesiones a los siete días. No se comparten datos con terceros salvo las llamadas a OpenRouter, que el usuario configura con su propia clave, y a Open Food Facts y Wger, que reciben solo un código de barras o un término de búsqueda.

8.8 Limitaciones conocidas

No hay diferenciación de roles: todos los usuarios autenticados tienen el mismo nivel de acceso. El catálogo de alimentos y ejercicios es compartido, sin un subconjunto privado por usuario más allá de poder ocultar entradas. Los limitadores de tasa son en memoria, por lo que protegen una sola instancia; un despliegue con varias instancias requeriría centralizarlos. Estas limitaciones están dentro del alcance definido y se recogen como líneas futuras.

9. Conclusiones y líneas futuras

9.1 Valoración del proyecto

NutriFit es un sistema funcional y desplegado que permite registrar alimentos, comidas y ejercicios, calcular el gasto energético estimado, evaluar la dieta con inteligencia artificial y visualizar tendencias a lo largo del tiempo. Se sostiene sobre cuatro capas (frontend React, cliente JavaFX, backend Spring Boot y PostgreSQL) en las que cada componente tiene responsabilidades delimitadas y se comunica con el siguiente a través de interfaces bien definidas.

El sistema no es un prototipo: los dieciocho prefijos de endpoint están implementados, probados y desplegados en producción accesible públicamente. El esquema está gestionado por Flyway con veintiocho migraciones. Los 199 tests cubren la lógica de negocio relevante; los unitarios corren sin infraestructura y los de integración sobre un PostgreSQL real.

9.2 Aportación del sistema

NutriFit demuestra que es posible construir un sistema multicapa completo dentro del alcance de un proyecto formativo, partiendo de decisiones técnicas justificadas. Las más relevantes son: acceso a datos con JdbcTemplate sin ORM, que mantiene las consultas SQL visibles y trazables; autenticación con token opaco en base de datos, que hace el logout inmediato y real; cálculo del resumen diario con una sola consulta de agregación; integración de modelos de lenguaje sin acoplarse a un proveedor; seguridad repartida en varias defensas independientes; y manejo de errores centralizado con respuesta uniforme.

La separación entre clientes y backend tiene una consecuencia verificable: el backend puede arrancarse, probarse y depurarse sin ninguna interfaz. El cliente web y el de escritorio consumen exactamente la misma API.

9.3 Aprendizajes técnicos

Gestión del esquema con migraciones versionadas. Flyway obliga a tratar el esquema como algo explícito y versionado. Cada cambio requiere un script numerado; el estado de la base de datos en cualquier entorno es predecible y reproducible. Las dos últimas migraciones (retirar la lista de la compra y limpiar sesiones con un trigger) muestran que el esquema también se mantiene, no solo crece.

Tests con y sin infraestructura. Diseñar los servicios con interfaces de repositorio permite probarlos con mocks. Cuando hace falta validar el SQL real, Testcontainers levanta un

PostgreSQL de verdad. Las dos estrategias se complementan: rapidez en lo unitario, realismo en lo de integración.

Integración de LLMs en aplicaciones Java. Conectar un modelo de lenguaje no es difícil técnicamente, pero obliga a gestionar fallos que no existen en APIs predecibles: timeouts, respuestas malformadas, claves agotadas y variabilidad en el formato. La estrategia de reintento y la ejecución asíncrona del plan semanal y el detective son respuesta a eso.

Despliegue real frente a despliegue local. El arranque en frío de Render, los ajustes de CORS al cambiar la URL de Vercel y las diferencias de collation entre PostgreSQL local y en producción no se ven en local. El despliegue continuo desde el principio los expuso pronto.

9.4 Limitaciones del sistema actual

Las limitaciones más relevantes son el catálogo compartido sin subconjunto por usuario, la falta de roles, los limitadores de tasa en memoria (válidos para una sola instancia) y un cliente de escritorio que no llega a los módulos de IA más recientes. Todas están identificadas de forma explícita y son consecuencia del alcance, no de errores de diseño.

9.5 Líneas futuras de trabajo

- Llevar al cliente JavaFX los módulos que hoy solo están en la web: detective, plan semanal, tendencias y configuración de IA.
- Añadir control de acceso por usuario sobre el catálogo de alimentos, más allá de poder ocultar entradas.
- Centralizar la limitación de tasa (por ejemplo, en Redis) para soportar varias instancias del backend.
- Introducir roles y permisos para distinguir usuarios de administradores del catálogo.
- Ampliar los tests del frontend hasta cubrir los componentes de visualización más complejos.
- Añadir recordatorios de hidratación y registro de comidas, y explorar USDA FoodData Central como fuente adicional de alimentos.

9.6 Cierre

NutriFit es un sistema funcional dentro de su alcance definido. Sus decisiones de diseño están documentadas, sus limitaciones están identificadas y sus pruebas cubren la lógica de negocio. No es un sistema terminado en todos sus aspectos, pero sí uno del que se puede razonar con precisión: qué hace, qué no hace y por qué.

10. Bibliografía

Documentación oficial — Backend

- Spring Boot Reference Documentation (3.x). Broadcom / VMware.
<https://docs.spring.io/spring-boot/docs/current/reference/html/>
- Spring Framework — JdbcTemplate and JDBC Core Classes. <https://docs.spring.io/spring-framework/reference/data-access/jdbc/core.html>
- Spring Security Crypto — Password Encoding. <https://docs.spring.io/spring-security/reference/features/authentication/password-storage.html>
- Flyway Documentation — Migrations. Redgate. <https://documentation.redgate.com/flyway>
- PostgreSQL Documentation. PostgreSQL Global Development Group.
<https://www.postgresql.org/docs/>
- springdoc-openapi — OpenAPI 3 Library for Spring Boot. <https://springdoc.org/>
- Testcontainers for Java. <https://java.testcontainers.org/>

Documentación oficial — Frontend y cliente

- React Documentation. Meta Open Source. <https://react.dev/>
- TypeScript Handbook. Microsoft. <https://www.typescriptlang.org/docs/>
- Vite Documentation. <https://vitejs.dev/guide/>
- Tailwind CSS Documentation. <https://tailwindcss.com/docs/>
- Recharts. <https://recharts.org/>
- Vitest Documentation. <https://vitest.dev/guide/>
- OpenJFX Documentation — JavaFX 21. <https://openjfx.io/openjfx-docs/>

Tests, APIs externas e infraestructura

- JUnit 5 User Guide. <https://junit.org/junit5/docs/current/user-guide/>
- Mockito Documentation. <https://site.mockito.org/>
- AssertJ — Fluent assertions for Java. <https://assertj.github.io/doc/>
- OpenRouter Documentation. <https://openrouter.ai/docs>
- Open Food Facts API Documentation. <https://wiki.openfoodfacts.org/API>
- Wger API. <https://wger.de/en/software/api>
- GitHub Actions, Docker, Render y Vercel — documentación oficial de cada servicio.

Referencias nutricionales y fisiológicas

- Mifflin, M.D., St Jeor, S.T., Hill, L.A., Scott, B.J., Daugherty, S.A., Koh, Y.O. (1990). A new predictive equation for resting energy expenditure in healthy individuals. *American Journal of Clinical Nutrition*, 51(2), 241–247.
- Ainsworth, B.E., Haskell, W.L., Herrmann, S.D., et al. (2011). 2011 Compendium of Physical Activities. *Medicine & Science in Sports & Exercise*, 43(8), 1575–1581.

Herramientas, estándares y aplicaciones de referencia

- Leach, P., Mealling, M., Salz, R. (2005). RFC 4122 — A Universally Unique Identifier (UUID) URN Namespace. IETF.
- Provos, N., Mazières, D. (1999). A Future-Adaptable Password Scheme. USENIX.
- MyFitnessPal, Cronometer y Yazio — aplicaciones comerciales consultadas como referencia funcional.

Anexo A — Guía de puesta en marcha

A.1 Requisitos previos

Herramienta	Versión mínima	Uso
Java (JDK)	21	Backend y cliente JavaFX
Maven	3.8	Construcción y arranque
PostgreSQL	15+	Base de datos del backend
Docker	cualquiera	Tests de integración (Testcontainers)
Node.js	18+	Frontend React
Git	cualquiera	Clonación del repositorio

A.2 Preparación de la base de datos

```
CREATE DATABASE nutrifit;  
CREATE USER nutrifit WITH PASSWORD 'nutrifit123';  
GRANT ALL PRIVILEGES ON DATABASE nutrifit TO nutrifit;
```

No hace falta crear las tablas manualmente. Flyway las aplica al arrancar el backend.

A.3 Configuración del backend

Crear `backend/src/main/resources/application-local.properties` :

```
spring.datasource.url=jdbc:postgresql://localhost:5432/nutrifit  
spring.datasource.username=nutrifit  
spring.datasource.password=nutrifit123  
spring.datasource.driver-class-name=org.postgresql.Driver  
  
spring.flyway.enabled=true  
spring.flyway.locations=classpath:db/migration  
  
server.port=8080  
  
# Claves de OpenRouter para las funciones de IA  
openrouter.gemma.api.key=TU_CLAVE  
openrouter.deepseek.api.key=TU_CLAVE
```

Este archivo está excluido del control de versiones, por lo que las claves no se publican.

A.4 Arranque del backend

```
cd backend
mvn -DskipTests spring-boot:run "-Dspring-boot.run.profiles=local"
```

El backend está listo cuando el log muestra `Tomcat started on port 8080`. Swagger UI queda disponible en `http://localhost:8080/swagger-ui.html`.

A.5 Arranque del frontend

Crear `frontend/.env.local` con `VITE_API_URL=http://localhost:8080` y ejecutar:

```
cd frontend
npm install
npm run dev
```

El frontend arranca en `http://localhost:5173`.

A.6 Arranque del cliente JavaFX

```
cd client
mvn javafx:run
```

El cliente debe arrancarse después del backend. La URL del backend se lee de `config.properties` o de la variable de entorno `NUTRIFIT_BACKEND_URL`.

A.7 Ejecución de tests

```
# Backend (unitarios + integración con Testcontainers; requiere Docker)
cd backend && mvn test

# Cliente JavaFX
cd client && mvn test

# Frontend
cd frontend && npm test
```

A.8 Problemas frecuentes

UnsupportedClassVersionError. El proyecto requiere Java 21. Comprobar con `java -version`.

El backend tarda 30-60 segundos en producción. El plan gratuito de Render suspende la instancia tras quince minutos de inactividad; la primera petición la reanuda.

Error de CORS en el frontend. El backend solo acepta el dominio de Vercel y `localhost:5173`. Si el frontend está en otra URL, actualizar los orígenes en `WebMvcConfig`.

Migration checksum mismatch. La estrategia de arranque ejecuta `repair()` antes de `migrate()`, por lo que en condiciones normales no aparece. Si se modificó una migración ya aplicada, la reparación la corrige en el siguiente arranque.

Anexo B — Referencia de endpoints de la API

El servidor escucha en `http://localhost:8080` en desarrollo y en `https://nutrifit-backend-ndoj.onrender.com` en producción. Todos los cuerpos usan `application/json`. La autenticación viaja en la cookie `nf_session` o en la cabecera `Authorization`. Cualquier error devuelve `{ timestamp, status, error, message, path }`. Documentación interactiva en `/swagger-ui.html`.

B.1 Autenticación — /api/auth

Método	Ruta	Descripción	Éxito
POST	/api/auth/register	Registra usuario y abre sesión	201
POST	/api/auth/login	Autentica usuario existente	200
POST	/api/auth/logout	Invalida el token de sesión activo	204

B.2 Alimentos y escáner

Método	Ruta	Descripción	Éxito	Errores
GET	/api/alimentos	Catálogo completo o filtrado por <code>?q=</code>	200	—
GET	/api/alimentos/{id}	Alimento por id	200	404
POST	/api/alimentos	Crea alimento	201	400
PUT	/api/alimentos/{id}	Actualiza alimento	200	400, 404
DELETE	/api/alimentos/{id}	Elimina alimento	204	404, 409
GET	/api/alimentos/externo	Búsqueda en Open Food Facts	200	—
POST	/api/alimentos/escanear-foto	Analiza foto del producto con IA	200	400, 429
GET	/api/escaner/{barcode}	Busca por código EAN en Open Food Facts	200	404

B.3 Comidas — /api/comidas

Método	Ruta	Descripción	Éxito	Errores
GET	/api/comidas	Comidas de un usuario por fecha	200	401
POST	/api/comidas	Crea comida	201	400, 401
DELETE	/api/comidas/{id}	Elimina comida y sus ítems en cascada	204	401, 404
GET	/api/comidas/{id}/items	Ítems con valores nutricionales calculados	200	404

POST	/api/comidas/{id}/items	Añade ítem a comida	201	400, 404
DELETE	/api/comidas/{cId}/items/{iId}	Elimina ítem de comida	204	404

B.4 Resumen diario, perfil y evaluación IA

Método	Ruta	Descripción	Éxito	Errores
GET	/api/resumen-diario	Totales del día (ceros si no hay comidas)	200	401
POST	/api/resumen/evaluacion-ia	Evaluación del resumen diario con IA	200	401, 429, 503
GET	/api/perfil/{id}	Perfil con TMB y TDEE calculados	200	401, 404
PUT	/api/perfil/{id}	Actualiza perfil; recalcula TMB y TDEE	200	400, 401

B.5 Ejercicios

Método	Ruta	Descripción	Éxito	Errores
GET	/api/ejercicios	Catálogo (filtros ?q= , ?tipo=)	200	—
GET	/api/ejercicios/externo	Búsqueda en Wger	200	—
GET	/api/ejercicios-registro	Registros de un usuario por fecha	200	401
POST	/api/ejercicios-registro	Registra una sesión de ejercicio	201	400, 401
DELETE	/api/ejercicios-registro/{id}	Elimina un registro propio	204	401, 404
GET	/api/ejercicios-registro/recuperacion	Sugerencia post-ejercicio intensivo	200	401

B.6 Módulos adicionales

Método	Ruta	Descripción	Éxito
GET/POST/DELETE	/api/hidratacion	Registro diario de hidratación	200/201/204
GET/POST/DELETE	/api/peso-historial	Historial de peso corporal	200/201/204
GET	/api/gamificacion	Racha y NutriScore del día	200
GET	/api/tendencias	Series de los últimos 90 días	200
GET/POST/DELETE	/api/plan-semanal	Plan de comidas generado por IA (asíncrono)	200/201/204
GET/POST	/api/detective	Análisis de hábitos (asíncrono)	200/202
GET/POST/DELETE	/api/retos	Catálogo de retos y progreso del usuario	200/201/204

GET/PUT/POST/DELETE	/api/ia-config	Configuración de IA del usuario (clave enmascarada)	200/204
GET	/api/health	Health check del backend	200